

Big Data –

♦ What is Big Data?

- Data explosion: from MBs to PBs every second.
- Simple definition: *“Big Data is data that’s too large or complex for traditional systems.”*
- Introduce 5 Vs:
 - Volume: Enormous scale of data (social media, IoT sensors)
 - Velocity: Data streaming in real-time (stock markets, sensors)
 - Variety: Text, images, audio, logs, sensor data
 - Veracity: Trustworthiness, noise, bias
 - Value: Turning data into insights

♦ Evolution of Big Data

Era	Characteristics	Example
1980s–1990s	Structured data in relational databases	Banking, Inventory
2000–2010	Web & social media data explosion	Google, Facebook
2010–2020	Hadoop, Spark revolution	MapReduce-based systems
2020–Present	Cloud + AI-driven data platforms	AWS, Azure, Databricks

◆ Beyond the Hype

- Hype: “Big Data will solve everything.”
- Reality: Big Data = potential only when combined with analytics.
- Gartner’s Hype Cycle: shows technologies maturing over time.
- Big Data today is part of AI-Data-Cloud ecosystem.

The Five Stages of the Hype Cycle

1. Innovation Trigger

- A new technology breakthrough or concept emerges.
- Early proof-of-concept stories and media interest spark curiosity.
- There may be no usable products yet, and commercial viability is unproven.
Example: Quantum computing in early research stages.

2. Peak of Inflated Expectations

- The technology receives massive publicity and hype.
- Some success stories appear, but many projects fail.
- Expectations often exceed reality.
Example: Initial buzz around blockchain or metaverse.

3. Trough of Disillusionment

- Interest declines as experiments and implementations fail to deliver.
- Producers of the technology shake out or fail.

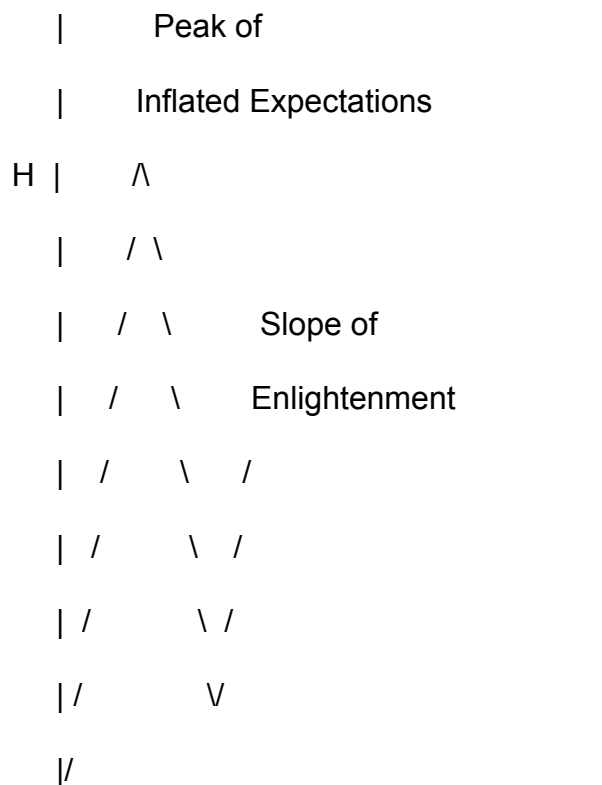
- Only those who truly understand the potential continue investing.
Example: Many AI startups post-2018 hype died off here.

4. Slope of Enlightenment

- More practical use cases start to emerge.
- Second- and third-generation products improve.
- Methodologies and best practices develop.
Example: Cloud computing and AI tools today.

5. Plateau of Productivity

- The technology becomes mainstream and productive.
- Its benefits are well-understood, and adoption becomes widespread.
Example: Mobile apps, cloud storage, machine learning in production.



Innovation Trough of Plateau of
Trigger Disillusionment Productivity

Identify examples of Big Data in:

1. E-commerce
2. Healthcare
3. Banking
4. Smart Cities

2 Big Data Skills and Sources

♦ Core Big Data Skill Areas

Skill Category	Description	Tools/Examples
Programming	Scripting and automation	Python, Scala, Java
Data Engineering	Collecting, cleaning, storing data	Hadoop, Spark, Kafka
Data Science & Analytics	Extracting insights, building models	Pandas, PySpark, MLlib
Visualization	Presenting insights visually	Power BI, Tableau
Cloud Platforms	Storing and managing large data	AWS, Azure, GCP
Soft Skills	Communication, domain understanding	Business sense

“What’s the difference between a *data engineer*, *data analyst*, and *data scientist*?”

♦ Sources of Big Data

Source	Example	Data Type
Social Media	Twitter, Instagram	Text, images
Sensors/IoT	Smart meters, vehicles	Time-series
Transactions	Banking, POS systems	Structured
Logs & Clickstreams	Website activity	Semi-structured
Healthcare	MRI, ECG, genomics	Unstructured
Public Datasets	Kaggle, Govt. Open Data	Mixed

Example dataset:

3Big Data Adoption, Research, and Changing Nature of Repositories

♦ Big Data Adoption Trends

- Early adopters: Tech giants (Google, Amazon, Netflix)
- Now pervasive: Manufacturing, Healthcare, Education, Government
- Drivers: Cloud computing, cheaper storage, AI integration
- Frameworks: Hadoop → Spark → Cloud-native (Databricks, Snowflake)

Case Study: Netflix

- Collects user viewing data (~1 petabyte/day)
- Uses Spark for real-time recommendations
- Demonstrates value of *data-driven decisions*

♦ Research in Big Data

- Focus on Scalability, Privacy, Real-time analytics, Ethics
- Emerging research areas:
 - Federated Learning
 - Data Governance

- Green Data (energy-efficient systems)
- Synthetic Data Generation

◆ Changing Nature of Data Repositories

Era	Storage System	Example
Pre-2010	RDBMS	Oracle, MySQL
2010–2015	Hadoop Distributed File System	HDFS
2015–2020	NoSQL	MongoDB, Cassandra
Now	Cloud + Lakehouse	AWS S3, Delta Lake, BigQuery

Introduction to ETL (Extract – Transform – Load)

1. What is ETL?

ETL stands for Extract, Transform, and Load —

a process used to move data from multiple sources into a centralized data warehouse or data lake for analysis and reporting.

It is a core concept in data engineering and big data pipelines.

2. ETL Process Overview

Step	Description	Example
Extract	Pulling data from various source systems such as databases, APIs, flat files, IoT streams, or cloud apps.	Extracting sales data from MySQL and customer data from Salesforce.
Transform	Cleaning, enriching, and converting extracted data into a usable format. Includes filtering, joining, aggregation, and validation.	Converting date formats, removing duplicates, calculating total revenue.
Load	Storing the transformed data into a target system such as a data warehouse, data lake, or analytics platform.	Loading final data into Amazon Redshift or Hadoop HDFS.

3. ETL Architecture

Components:

- 1. Source Layer: Systems generating data (ERP, CRM, IoT devices, etc.).
 - 2. Staging Layer: Temporary storage for extracted raw data.
 - 3. Transformation Layer: Business logic applied here.
 - 4. Target Layer: Final structured data warehouse for analytics.
-

4. Common ETL Tools

Type	Tools / Platforms
Traditional / On-prem	Informatica PowerCenter, IBM DataStage, Talend, Pentaho
Cloud-based	AWS Glue, Azure Data Factory, Google Dataflow

Open-source	Apache NiFi, Apache Airflow (for orchestration), Luigi
Big Data ETL	Spark ETL, Hadoop MapReduce, Flink, Databricks

5. Common Data Transformations

- Data cleansing (remove nulls, duplicates)
 - Type conversion (string → date, float → int)
 - Aggregation (sum, avg, min, max)
 - Joining data from multiple sources
 - Normalization / Denormalization
 - Business rule application (e.g., apply tax rate, discount logic)
-

6. ETL vs ELT

Aspect	ETL (Extract → Transform → Load)	ELT (Extract → Load → Transform)
Processing	Transformation before loading	Transformation after loading
Target System	Data Warehouse	Data Lake / Cloud DB
Use Case	Structured, smaller data	Big Data, cloud-native pipelines
Example	Informatica, Talend	Snowflake, BigQuery, Databricks

7. ETL in the Big Data Ecosystem

- In Hadoop or Spark, ETL can be implemented as:
 - E: Load data into HDFS or S3 from multiple sources.

- T: Use MapReduce, Hive, or Spark to clean and transform.
 - L: Store back into HDFS, Hive tables, or data warehouse.
 - Example:
 - Logs extracted from web servers → cleaned in Spark → loaded into Hive for BI analysis.
-

8. Benefits of ETL

- Centralized & consistent data.
 - Improved data quality and reliability.
 - Enables faster reporting and analytics.
 - Historical data storage.
 - Scalability with automation and cloud tools.
-

9. Challenges in ETL

- Handling schema changes in source systems.
 - Large data volume and performance optimization.
 - Real-time vs batch processing requirements.
 - Maintaining data lineage and audit trails.
 - Error handling and reprocessing.
-

10. Real-World Example

Scenario:

An e-commerce company wants to analyze sales trends.

- Extract: Pull order data from MySQL, user data from MongoDB, and ad spend from Google Ads API.
- Transform: Clean data, standardize currencies, calculate profit margin.
- Load: Store final dataset in AWS Redshift for Power BI dashboarding.

11. Summary

Step	Description
E	Collect raw data
T	Clean, format, and enrich it
L	Store in target data store
Goal	Enable reliable analytics and reporting

DATA ENGINEERING

1What Is Data Engineering?

Definition:

Data Engineering is the discipline of designing, developing, and maintaining systems that enable the collection, storage, processing, and accessibility of data at scale.

It ensures data is:

- Accurate (clean & reliable)
- Accessible (available to users & systems)
- Actionable (ready for analytics, AI & decision-making)

Example:

An e-commerce platform collecting millions of transactions daily — data engineers design pipelines to move, clean, and prepare that data for analysis in dashboards or ML models.

2Key Components of Data Engineering

2.1 Data Collection

- Process of gathering raw data from multiple sources such as:
 - Databases (MySQL, PostgreSQL)
 - APIs
 - IoT devices
 - Web logs
 - Cloud applications (Salesforce, Shopify)
 - Tools: Apache Nifi, Kafka, AWS Kinesis, Flume
 - Goal: Collect data in real-time or batches with minimal loss.
-

2.2 Data Storage

- After collection, data must be stored efficiently for scalability and reliability.

- Types of Storage:
 - Data Lakes (raw/unstructured data) – e.g., Hadoop HDFS, Amazon S3, Azure Data Lake
 - Data Warehouses (structured, query-optimized data) – e.g., Snowflake, BigQuery, Redshift
 - Databases (operational storage) – e.g., MySQL, MongoDB
 - Considerations: scalability, cost, data security, and performance.
-

2.3 Data Processing

- Raw data needs transformation into usable formats.
 - Processing types:
 - Batch Processing: Process large chunks of data periodically. (Hadoop MapReduce, Spark)
 - Stream Processing: Process data in real-time as it arrives. (Kafka, Flink, Spark Streaming)
 - Tasks: Cleaning, transformation, aggregation, validation.
-

2.4 Data Pipelines

- A data pipeline automates the flow of data from source to destination.
- Includes:
 - Extraction
 - Transformation

- Loading (ETL/ELT)
 - Tools: Apache Airflow, Luigi, AWS Glue, Azure Data Factory.
 - Goal: Reliable, automated, and reusable data workflows.
-

2.5 Data Quality and Governance

- Ensures data is accurate, consistent, and compliant.
 - Data Quality Aspects:
 - Completeness
 - Accuracy
 - Timeliness
 - Consistency
 - Governance: Policies and tools for managing access, security, and lineage.
 - Tools: Great Expectations, Collibra, Apache Atlas.
-

3 Why Is Data Engineering Important?

- Data is the new oil — but it's only valuable when refined.
- Data engineering makes raw data usable by:
 - Enabling data-driven decisions.
 - Supporting AI & analytics workloads.

- Maintaining scalability as data grows.
- Reducing time from data generation → insight.
- Ensuring data reliability & compliance.

Example:

Netflix uses strong data pipelines to deliver personalized recommendations to millions of users daily.

4Core Responsibilities of a Data Engineer

Area	Description
Data Pipeline Development	Build & maintain ETL/ELT pipelines
Data Modeling	Design efficient schemas for warehouses
Data Integration	Combine multiple data sources
Performance Optimization	Optimize storage and processing speed
Automation & Scheduling	Manage workflows (Airflow, Cron, etc.)
Monitoring & Maintenance	Detect pipeline failures, ensure uptime
Security & Compliance	Manage data access & governance rules

5Why Does Data Need Processing through Data Engineering?

- Raw data = messy, inconsistent, incomplete.

- Processing is needed to:
 1. Clean errors, duplicates, and null values.
 2. Standardize formats (dates, currencies, etc.).
 3. Integrate data from multiple systems.
 4. Apply business logic (sales → profit, etc.).
 5. Enable accurate reporting and analytics.

Without data engineering, analysts spend 70–80% of time cleaning data instead of analyzing it.

6 Tools and Technologies in Data Engineering

Category	Tools / Platforms
Data Collection / Ingestion	Apache Kafka, Flume, Sqoop, NiFi, AWS Kinesis
Data Storage	Hadoop HDFS, Amazon S3, Azure Data Lake, Google Cloud Storage
Data Processing	Apache Spark, Hadoop MapReduce, Apache Flink
Workflow Orchestration	Apache Airflow, Luigi, Oozie
Data Warehousing	Snowflake, BigQuery, Redshift
Data Quality & Governance	Great Expectations, Apache Atlas, Collibra
Monitoring & Logging	Prometheus, Grafana, ELK Stack
Programming Languages	Python, Java, Scala, SQL

7 Challenges in Data Engineering

Challenge	Description
Data Volume	Managing terabytes to petabytes efficiently
Data Variety	Handling structured, semi-structured, unstructured data
Data Velocity	Real-time streaming vs batch ingestion
Data Quality	Maintaining accuracy & consistency
Scalability	Systems must scale horizontally
Security & Privacy	Compliance with GDPR, HIPAA, etc.
Cost Management	Cloud storage & processing cost control
Tool Integration	Managing diverse technologies in one ecosystem

8 Data Engineering vs. Data Science

Aspect	Data Engineering	Data Science
Focus	Building systems to move, store & process data	Analyzing and modeling data for insights
Goal	Prepare clean, structured data	Generate business insights, predictions
Core Skills	SQL, Hadoop, Spark, ETL, Airflow	Statistics, ML, Python, R, Visualization
Tools	Kafka, Airflow, Spark, Hive	TensorFlow, Pandas, scikit-learn
Output	Data pipelines, warehouses, APIs	Reports, dashboards, ML models
Dependency	Prepares data for data scientists	Uses data engineered by DEs

9 Future Trends in Data Engineering

- DataOps & MLOps – applying DevOps principles to data pipelines.
- Cloud-native Data Engineering – AWS Glue, Azure Synapse, GCP Dataflow.
- Real-time Analytics – Streaming pipelines using Kafka + Flink.
- Serverless Data Pipelines – pay-per-use architecture (AWS Lambda).
- Data Lakehouse Architecture – combining warehouse + lake (Databricks Delta Lake, Snowflake).
- AI-assisted Data Engineering – automated data cleaning & schema detection.

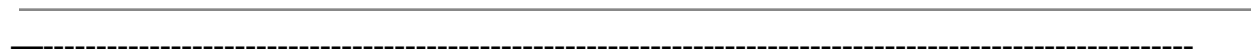
Conclusion

Data Engineering is the backbone of any data-driven organization.

It transforms raw, scattered data into a trusted, structured foundation for analytics, AI, and decision-making.

A skilled data engineer ensures:

- Data is reliable
- Pipelines are efficient
- Systems are scalable
→ enabling true data-driven success.



Topic: Hadoop – Architecture, Components, and Evolution

1. Introduction to Hadoop

- What is Hadoop?
 - An open-source framework for distributed storage and processing of large datasets.
 - Developed by Doug Cutting and Mike Cafarella, inspired by Google File System (GFS) and MapReduce papers.
 - Part of the Apache ecosystem.
 - Written in Java.
 - Core goal: process huge volumes of data efficiently across clusters of commodity hardware.
-

2. Why Hadoop?

- Traditional RDBMS systems fail for:
 - Volume, Variety, Velocity (Big Data 3Vs).
 - Scalability limits.
 - High storage cost.
- Hadoop provides:
 - Scalability – add more nodes easily.
 - Fault tolerance – automatic recovery from node failure.
 - Cost efficiency – runs on commodity hardware.

- Parallel processing – through MapReduce.
-

3. Core Components of Hadoop

Component	Description
HDFS (Hadoop Distributed File System)	Storage layer – stores large files across multiple machines.
YARN (Yet Another Resource Negotiator)	Cluster management layer – handles resource allocation & job scheduling.
MapReduce	Data processing layer – programming model for distributed computation.
Common Utilities	Libraries and Java utilities for Hadoop operations.

4. Hadoop Architecture (Overview)

- Master-Slave Architecture
 - HDFS → NameNode (Master), DataNode (Slave)
 - MapReduce → JobTracker (Master), TaskTracker (Slave)
 - YARN → ResourceManager (Master), NodeManager (Slave)
 - Data is split into blocks (default: 128MB in Hadoop 2.x, 256MB in Hadoop 3.x).
 - Blocks replicated (default: 3 copies).
-

5. Hadoop Ecosystem Components (Brief)

- Pig → Scripting platform for large-scale data processing.

- Hive → SQL-like data warehouse system.
 - HBase → NoSQL database built on HDFS.
 - Sqoop → Data transfer between Hadoop & RDBMS.
 - Flume → Ingest streaming logs.
 - Oozie → Workflow scheduler.
 - Zookeeper → Coordination service.
 - Spark → In-memory big data processing.
-

-- HDFS (Hadoop Distributed File System)

♦ 1. Introduction to HDFS

HDFS stands for Hadoop Distributed File System — a scalable, fault-tolerant, distributed storage system designed to store and manage large datasets across clusters of commodity hardware.

- Inspired by Google File System (GFS).
- Handles large files (GBs–TBs).
- Provides high throughput, not low latency.
- Optimized for write once, read many workloads.

♦ 2. HDFS Architecture

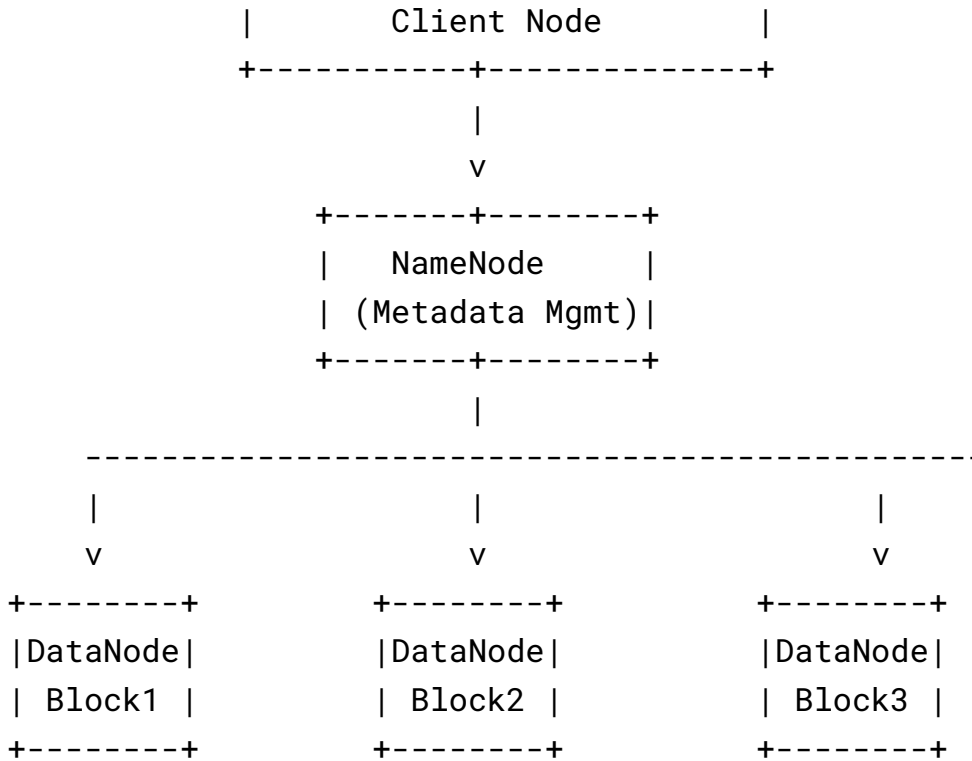
Master–Slave Architecture

HDFS follows a master-slave design consisting of two main daemons:

Component	Role
NameNode (Master)	Manages the file system namespace and metadata (directory structure, permissions, block locations).
DataNode (Slave)	Stores actual data blocks. Handles read/write requests from clients and performs block creation, deletion, and replication.
Secondary NameNode	Not a backup; periodically merges edit logs and FsImage to reduce NameNode load.

Architecture Diagram (Conceptual)

+-----+



Key Points:

- Files are split into blocks (default: 64 MB → 128 MB → 256 MB depending on version).
- Each block is replicated (default: 3 copies).
- NameNode knows *where* blocks are, but doesn't store them.
- DataNodes store actual data and send heartbeats to NameNode.

♦ 3. HDFS Workflow

Let's see how reading and writing happens step-by-step 🙌

Write Operation Workflow

1. Client contacts NameNode → requests file creation.
 2. NameNode checks namespace, grants permission.
 3. File is split into blocks (e.g., 3 blocks).
 4. NameNode assigns DataNodes for each block.
 5. Client writes blocks → DataNodes store them → replicated automatically.
 6. After successful write, NameNode updates metadata.
-

Read Operation Workflow

1. Client asks NameNode for block locations of the requested file.
 2. NameNode responds with DataNode addresses.
 3. Client reads data directly from nearest DataNodes.
 4. File is reconstructed in order (Block1 → Block2 → Block3).
-

Example

File: `customer_data.csv` (300 MB)

- Block size: 128 MB
 - Replication factor: 3
-

4. Difference Between Traditional File Systems (NTFS, FAT, etc.) and HDFS

Feature	Traditional File Systems (NTFS, FAT, ext4)	HDFS (Hadoop Distributed File System)
---------	---	--

Storage Type	Local storage (single system)	Distributed across many nodes
Scalability	Limited to single disk or server	Highly scalable (add more nodes easily)
Fault Tolerance	Hardware failure = data loss	Automatic block replication prevents data loss
Data Size Handling	Best for small to medium files	Optimized for large files (GBs–TBs)
Access Speed	Low latency for small files	High throughput for large data processing
Write Operation	Random read/write supported	Write-once, read-many model
Data Replication	Manual backup required	Automatic replication managed by NameNode
Hardware Requirement	Expensive, reliable servers	Commodity, low-cost hardware
File Metadata	Stored locally	Managed by centralized NameNode
Usage	OS-level file storage	Big Data analytics platforms

5. Version-wise Changes in HDFS (Hadoop 1.x → 2.x → 3.x)

Feature / Version	HDFS 1.x	HDFS 2.x	HDFS 3.x
Architecture	Single NameNode (SPOF)	Federation: Multiple NameNodes	Enhanced HA with multiple Standby NameNodes
High Availability (HA)	No HA	Added Active/Standby NameNodes	Improved failover and automatic recovery
Federation	Not available	Supported	Improved scalability

Storage Efficiency	Replication (3 copies)	Replication	Erasure Coding (saves ~50% space)
Default Block Size	64 MB	128 MB	256 MB
Scalability	Limited	Improved	Highly scalable with better balancing
DataNode Rebalancer	Basic	Improved	Intra-Node Balancer introduced
YARN Integration	Not present	Introduced	Optimized
Container/Docker Support		Limited	Supported
Fault Recovery	Manual restart	Automatic failover	Advanced recovery mechanisms

6. Key Advantages of HDFS

- Fault Tolerance through replication
- Scalability – add nodes easily
- Cost-effective – runs on commodity hardware
- High throughput data access

Compatibility with big data frameworks (MapReduce, Spark, Hive, etc.)

7. Summary

Term	Description
NameNode	Manages metadata (file names, block locations)
DataNode	Stores blocks of data

Secondary NameNode	Merges FsImage + EditLogs
Block	Logical chunk of file (64–256 MB)
Replication	Multiple copies for fault tolerance
Client	Reads/Writes data using HDFS API

Quick Recap

- HDFS is designed for big data, not small files.
- Files are split → replicated → stored across nodes.
- Versions evolved from single NameNode (v1) → Federation & HA (v2) → Erasure Coding + Cloud/Container support (v3).
- Compared to NTFS/ext4, HDFS is distributed, fault-tolerant, and horizontally scalable.

-
- Hadoop 1.x:
 - One NameNode → single point of failure.
 - Used Secondary NameNode for periodic checkpoints.
 - No fault tolerance at the NameNode level.
 - Hadoop 2.x:
 - Introduced High Availability (HA) and Federation.
 - Added Active / Standby NameNode mechanism.
 - Removed the single point of failure issue.

- Hadoop 3.x:
 - Enhanced HA with multiple standby NameNodes.
 - Integrated Erasure Coding for efficient storage.
 - Optimized for large-scale and cloud-ready deployments.
-

Example (HDFS HA setup)

In Hadoop 2.x/3.x:

- One Active NameNode handles all client requests.
 - One or more Standby NameNodes stay synchronized via JournalNodes.
 - If the Active NN fails, Zookeeper automatically promotes a Standby NN to Active.
-
- -----

What Is Stored Inside the NameNode (in HDFS)

The NameNode is the master node of HDFS — it does not store actual file data, but it stores metadata — i.e., information *about* the data.

1. Metadata Stored in NameNode

Type of Information	Description
File and Directory Structure	Hierarchical namespace — like a directory tree of all files and folders in HDFS.

File-to-Block Mapping	Which file is split into which blocks (e.g., <code>/user/data.txt</code> → <code>block_001</code> , <code>block_002...</code>).
Block-to-DataNode Mapping	Which DataNodes store each block (e.g., <code>block_001</code> → <code>DataNode3</code> , <code>DataNode7</code> , <code>DataNode8</code>).
Replication Information	The replication factor for each block (e.g., 3 copies per block).
Permissions & Ownership	POSIX-style access permissions: owner, group, read/write/execute flags.
Timestamps	File creation, modification, and access times.
Quota Information	Directory-level quotas — maximum number of files or total storage allowed.
Namespace ID and Cluster ID	Unique identifiers to distinguish HDFS clusters and their namespaces.
Edit Logs & Checkpoints (FsImage + EditLog)	Persistent metadata storage on disk — described below.

2. Files Maintained by the NameNode

Inside the NameNode's local file system (usually in `/dfs/nn/current/`), two main files store this metadata permanently:

File Name	Purpose
FsImage	Snapshot of the entire HDFS namespace at a given time (like a full backup of metadata).
EditLog	Records all recent HDFS changes (like transactions — create, delete, rename, etc.).

During startup:

- NameNode loads FsImage into memory.
 - Then applies all changes from EditLog.
 - This creates the current in-memory state of the filesystem.
-

3. What's *Not* Stored in the NameNode

Not Stored	Stored In
Actual file data blocks	DataNodes
Replicated copies of data	DataNodes
Temporary computation data	YARN / other frameworks

4. Memory Requirement

All metadata is kept in RAM for fast access.

Hence, NameNode needs large memory — roughly 1 GB of RAM per 1 million files/blocks (approximation).

That's why Hadoop 1.x faced scalability issues with very large clusters.
